

Design and Implementation of a Precise Timeline-Driven Scheduler for FreeRTOS

Academic Report: Project Assignment 2026

Lombardozzi, Christian Masoumkhani, Amin Papantuono, Federico
Pedron, Marco Stella, Paolo

Abstract

Deterministic task scheduling is a fundamental requirement in safety-critical embedded systems, where unpredictable execution timing can lead to catastrophic failures. Standard priority-based operating system kernels, such as FreeRTOS [1], are not inherently designed to satisfy the strict temporal guarantees demanded by *Hard Real-Time* (HRT) environments. This report presents the design and implementation of a *Time-Triggered* (TT) timeline scheduler developed as a non-invasive extension to the FreeRTOS kernel, carried out by Group 49 within the *Operating Systems for Embedded Systems* course at Politecnico di Torino for the 2025/26 academic year, in compliance with the official project specification [2]. The proposed architecture replaces the standard priority-based dispatching model with a deterministic, compile-time-defined schedule organized around a repeating *major frame* subdivided into *minor frames*. HRT tasks are assigned to exclusive, non-overlapping temporal slots and are enforced through a *fail-stop* policy upon *deadline* violation. *Soft Real-Time* (SRT) tasks are dispatched opportunistically during idle time under a *best-effort* policy. The entire system operates exclusively on statically allocated memory, eradicating *heap* fragmentation and non-deterministic allocation latency. The implementation targets the *Quick Emulator* (QEMU) Cortex-M3 platform and is validated against two quantitative non-functional requirements: a release *jitter* bounded to at most one system tick, and a scheduler-induced *overhead* not exceeding 10% of total *Central Processing Unit* (CPU) utilization. A comprehensive automated testing framework, built in Python, orchestrates the full validation pipeline—from firmware compilation to *Universal Asynchronous Receiver-Transmitter* (UART) log analysis—and produces cycle-accurate jitter and overhead metrics to formally verify compliance with these constraints.

Contents

1 Architectural Overview and System Initialization	2
1.1 Design Requirements and Operational Objectives	2
1.2 System Configuration and Parsing Phase	3
1.3 Data Structures and Internal Representation	3
1.4 Application Programming Interfaces (APIs)	4
1.5 Sorting Algorithms and Execution Priorities	4
1.6 Temporal Validation and Fault-Tolerance Policies	4
1.7 Queue Management and Soft Real-Time Execution	5
1.8 Workload Simulation and Execution Dynamics	5
1.9 Context Restoration and Cyclic Reset	6
2 Memory Management and Static Allocation	6
2.1 Allocation Strategies: Dynamic vs. Static	6
2.2 Task-Specific Memory Architecture	7
3 Environment Setup and Deterministic Emulation	7
3.1 Software Architecture and Repository Layout	7
3.2 Cross-Platform Portability and Reproducibility	7
3.3 Virtual Clock Configuration and Determinism	8
4 Kernel Integration and Scheduling Hooks	8
4.1 Execution State Management and Transition	8
4.2 Execution Flow Interception and Context Switching	9
4.3 Deterministic Dispatching and Amortized Access	9
5 Telemetry System and Asynchronous Tracing	10
5.1 Asynchronous Event Logging and Buffer Management	10
5.2 Offline Analysis and Telemetry Integration	10
6 Validation Framework and Automated Testing	10
6.1 Pipeline Orchestration and Test Execution	10

6.2 Log Parsing and Frame Boundary Detection	11
6.3 Metrics Extraction and Jitter Analysis	11
6.4 Visualization Generation and Regression Checking	11

7 Concluding Remarks and Final Assessment **12**

1. Architectural Overview and System Initialization

The implemented architecture fundamentally alters the default task management mechanisms of the operating system by enforcing a deterministic dispatching model. This section delineates the core design principles, the logical data structures, and the operational lifecycle of the scheduling engine.

The scheduler implements a time-driven logic, establishing a scheduling paradigm known in the literature as *timeline scheduling*. The scheduled tasks are executed cyclically in a strictly predictable and repetitive manner within the boundaries of a global temporal window defined as the major frame.

The adopted TT architecture ensures that HRT tasks are confined to precise temporal slots, denoted as minor frames. Consequently, their start and end execution instants—where the latter represents the absolute deadline—are strictly enforced, guaranteeing a deterministic behavior that is consistently reproduced at every cycle. Furthermore, the architecture ensures that SRT tasks, whose exact start and end times are not known a priori, are dispatched during the system’s idle time. These tasks consume all available processor execution time until their completion; however, their completion within the current major frame is not guaranteed, as they are managed under a best-effort policy.

1.1. Design Requirements and Operational Objectives

To satisfy the rigorous constraints of real-time embedded environments, the implementation enforces a set of non-functional requirements centered on predictability and efficiency. These objectives strictly adhere to the official project specifications established for the course [2].

Specifically, deterministic execution is guaranteed through the adoption of the aforementioned major frame and minor frame scheduling hierarchy. Temporal precision is maintained by bounding the release jitter to a maximum of one system tick, while computational impact is strictly regulated to ensure that scheduler-induced overhead does not exceed 10% of the total CPU utilization.

1.2. System Configuration and Parsing Phase

The initial phase of the system’s lifecycle involves the generation of tasks based on a user-defined configuration. Specifically, the operator is permitted to provide one or more configurations through a file formatted in *JavaScript Object Notation* (JSON)¹; these directives are subsequently translated at compile-time into an array of statically allocated data structures, thereby ensuring predictability and completely eradicating memory fragmentation issues.

1.3. Data Structures and Internal Representation

The fundamental logical block for task representation is constituted by the `TimelineTaskConfig_t` structure. It is composed of a series of fields shared between HRT tasks and SRT tasks, flanked by specific parameters that assume semantic value exclusively for the HRT domain; in the event that the structure describes an SRT task, these specific fields are initialized to safe default values.

Listing 1: Definition of the data struct `TimelineTaskConfig_t`

```
typedef struct {
    const char * pcName;
    TaskFunction_t pvTaskFunction;
    TimelineTaskType_t xType;
    TaskHandle_t xTaskHandle;
    const void * pvTaskParams;
    configSTACK_DEPTH_TYPE xStackDepth;
    StaticTask_t * pxTaskBuffer;
    StackType_t * pxStackBuffer;
    uint32_t ulStart_time;
    uint32_t ulEnd_time;
    uint32_t ulSubframe_id;
    uint32_t ulMaxDelay;
    uint32_t ulMinDelay;
    ListItem_t xSRTListItem;
} TimelineTaskConfig_t;
```

Analyzing the common fields, the structure primarily presents the signature `const char * pcName`, which stores a descriptive name essential for tracing and debugging operations. This is followed by `TaskFunction_t pvTaskFunction`, which represents the pointer to the C function implementing the actual workload of the task, and `TimelineTaskType_t xType`, which defines the categorization of the task itself utilizing a dedicated enumerator. The general parameters are completed by `TaskHandle_t xTaskHandle`, delegated to the management of the instance within the kernel,

and `const void * pvTaskParams`, which provides the pointer to the task parameters.

A second grouping of common fields is entirely dedicated to the static memory allocation parameters, which play a critical role in the reinitialization procedures. Specifically, the definition incorporates `configSTACK_DEPTH_TYPE xStackDepth` to define the stack depth in *words*, flanked by `StaticTask_t * pxTaskBuffer`, which points to the statically allocated memory area for the *Task Control Block* (TCB), and `StackType_t * pxStackBuffer`, which refers to the memory area designated for the stack itself. It is imperative to emphasize that the entire architecture has been conceived to operate exclusively through static allocation, a design choice dictated by the necessity to maximize the reliability and determinism of the system, as will be explored comprehensively in Section 2.

Regarding the fields intended for the exclusive use of HRT tasks, the structure includes `uint32_t ulStart_time` and `uint32_t ulEnd_time`, which represent respectively the absolute activation tick and the absolute deadline, both calculated in relation to the boundaries of the major frame. To these is added `uint32_t ulSubframe_id`, a logical grouping identifier necessary to bind the task to a specific minor frame. Furthermore, fundamental for validation metrics are the variables `uint32_t ulMaxDelay` and `uint32_t ulMinDelay`, which record the maximum and minimum execution delays; these parameters are instrumental for the precise calculation of the task’s jitter, whose tolerable upper limit is strictly defined by the project requirements.

Finally, there is a field designated for the exclusive use of SRT tasks: `ListItem_t xSRTListItem`. This linked list node allows the insertion of the task within the ready queue specific to the SRT timeline, ensuring that each instance constantly maintains the reference to the static data structure that regulates its best-effort execution.

The adopted design approach dictates that the user is not required to manually populate the entire structure. Through the JSON file, the operator only needs to provide the essential parameters, such as `ulStart_time` and `ulEnd_time`. Derived fields, such as `ulSubframe_id`, are dynamically calculated and populated by the function `static BaseType_t xTimeline_InitAndValidateTasks(TimelineConfig_t * pxCfg)` during the validation phase, thus minimizing exposure to human errors. Since the JSON configuration file is processed by an external tool to generate the source file `task_generated.c` (containing the array of `TimelineTaskConfig_t` structures), the memory allocation is completely static and resolved at compile-time for any provided configuration.

¹JSON was selected for its native integration with the Python-based testing orchestrator, enabling seamless parsing and automated generation of C-language header structures.

The system nonetheless maintains a high degree of flexibility, allowing the user to dynamically define the stack size for each individual task directly from the configuration file, thus adapting to the specific requirements of the workload.

1.4. Application Programming Interfaces (APIs)

In accordance with the principle of encapsulation and information hiding, the interaction surface exposed to the user is intentionally limited to two primary APIs. The first is the function `const TimelineConfig_t * pxGetTimelineConfig( void )`, tasked with the extraction of the dynamically generated configuration arrays produced by the Python tool and their respective temporal limits. The second, and structurally more crucial, is `void vConfigureScheduler( const TimelineConfig_t * pxConfig )`, which takes charge of the comprehensive initialization of the entire time-triggered scheduling environment.

Delving into the operational detail, `vConfigureScheduler` orchestrates the complete startup sequence: it acquires the configuration, iterates over the array of `TimelineTaskConfig_t` structures, and delegates their ascending chronological sorting to the function `static void vSortTasksByStartTime( const TimelineConfig_t * pxConfig )`. Subsequently, it subjects the sorted array to the rigorous validation of `xTimeline_InitAndValidateTasks`. Having successfully passed these checks, it proceeds to the actual creation of the tasks at the kernel level by invoking the native API `BaseType_t xTaskCreateStatic( TaskFunction_t pxTaskCode, const char * const pcName, const uint32_t ulStackSize, void * const pvParameters, BaseType_t uxPriority, StackType_t * const puxStackBuffer, StaticTask_t * const pxTaskBuffer )`, which guarantees their static residence in memory.

As final preparatory steps, the function forces the suspension of all newly created tasks and systematically inserts the SRT tasks into their respective static queue (detailed in Section 1.7). Following this procedure, the scheduler is formally ready for execution. In the case of a positive outcome, the function `void prvTimeline_PrintSubFrames( const TimelineConfig_t * pxCfg )` provides the operator with a textual report of the loaded minor frame topology; conversely, in the case of an erroneous configuration, the startup process is actively interrupted, appropriately signaling the critical issues encountered. The decision to centralize these operations within `vConfigureScheduler` compartmentalizes the kernel's complexity, providing the developer with a single, secure, and controlled entry point.

1.5. Sorting Algorithms and Execution Priorities

The function `vSortTasksByStartTime` bears the responsibility of sorting the array of structures based on the predicted activation time. Since this metric is semantically valid only for HRT tasks, in SRT instances this field is conventionally initialized to zero. In order to optimize the search times for the next HRT task to be executed, the sorting algorithm has been designed to mathematically discriminate the nature of the task: when it encounters an SRT task, rather than evaluating its temporal field, it confines it to the terminal positions of the array. This operation ensures that, upon completion of the sorting, all HRT tasks are perfectly sequenced in ascending order of start time, while the SRT tasks form a contiguous block at the tail of the array, rigorously preserving the relative order originally defined by the user. This preservation is of vital importance: since the execution of SRT tasks is opportunistic and offers no guarantees of termination within the major frame, the user can exploit the ordering in the JSON file to define implicit execution priorities, placing the SRT tasks of greater relevance in the first positions. This positional segregation thus guarantees a clear and functional separation between the distinct domains.

To implement this logic, the *Insertion Sort* algorithm was selected. Although characterized by a quadratic asymptotic computational complexity, this algorithm proves to be the most appropriate engineering choice for several reasons². Firstly, the performance impact is negligible considering that the cardinality of the array in these embedded systems is typically small, and that the sorting operation is performed *one-off* exclusively during the initialization phase, prior to the scheduler's startup. Secondly, and an even more determining factor, Insertion Sort operates *in-place*, requiring no auxiliary memory allocation; this spatial parsimony constitutes an indispensable requirement in heavily constrained architectures. Finally, the intrinsic stability property of this algorithm mathematically guarantees the conservation of the initial relative order of the SRT tasks, thus satisfying the architectural requirement discussed previously.

1.6. Temporal Validation and Fault-Tolerance Policies

The mitigation of errors related to the malformation or the overlapping of the user-defined temporal intervals is delegated to the function

²For typical embedded real-time applications where the task cardinality N rarely exceeds a few dozens, the $\mathcal{O}(N^2)$ worst-case complexity of Insertion Sort is heavily outweighed by its minimal instruction cache footprint, lack of recursive stack allocations, and $\mathcal{O}(N)$ performance on nearly sorted data.

xTimeline_InitAndValidateTasks. This component performs an exhaustive static analysis on the configuration, raising exceptions if the following are detected: temporal slots with a negative duration (where the start time exceeds the end time), tasks whose boundaries exceed the duration of the major frame or the individual minor frame to which they belong, or, as a maximum criticality, tasks whose execution intervals temporally overlap.

Upon the occurrence of any of these violations, the error management policy dictates that the system immediately enters an intentional deadlock state even before the scheduler is started, emitting a detailed diagnostic log that requires manual corrective intervention. This design choice, formally defined as *fail-stop*³, was starkly preferred over the mere suppression of the malformed task. The theoretical reasons are manifold: in the event of temporal overlapping, the system does not possess the heuristics necessary to decide which of the two tasks should have semantic precedence. Furthermore, silently ignoring a task in a real-time environment—where the software frequently governs physical actuators and critical systems—could trigger catastrophic failures. The arbitrary elimination of a computational node could invalidate the data dependency chain, leading the entire system into undefined behavioral states.

1.7. Queue Management and Soft Real-Time Execution

The management of SRT tasks is orchestrated through a dedicated queueing mechanism that strictly enforces the designed temporal policies while completely avoiding dynamic memory allocation. At the core of this subsystem is the statically allocated `TimelineSrtState_t xSrtState` structure, which encapsulates a native FreeRTOS `List_t xReadyList`. To prevent heap fragmentation and comply with the rigorous robustness requirements of the scheduler, the individual list items, represented by the `ListItem_t xSRTListItem` members, are statically embedded directly within each task’s `TimelineTaskConfig_t` configuration matrix. This architectural choice ensures that the queue is fully constructed at compile time and initialized during the system boot phase without invoking any blocking or non-deterministic allocation functions.

The population of the SRT queue preserves the exact sequential order defined by the user in the initial configuration. This constraint is enforced during the initialization phase by the `vSortTasksByStartTime` function. This routine

³A fail-stop behavior ensures that the system fails safely by halting execution explicitly, rather than propagating Byzantine faults or unpredictable timing violations across the physical actuators governed by the firmware.

applies an in-place stable insertion sort that, while guaranteeing the absolute precedence of HRT tasks, meticulously maintains the relative declaration order of the SRT elements. Subsequently, the `void prvInitSRTQueue( void )` function iterates over this sorted array and appends each SRT task to the `xReadyList` utilizing the `vListInsertEnd` native FreeRTOS macro. This guarantees that the scheduling engine processes the tasks according to a strict *First-In, First-Out* (FIFO) policy.

By adopting the native FreeRTOS doubly linked list architecture, the scheduler achieves optimal $\mathcal{O}(1)$ access times for task selection. The retrieval of the next available task is performed by the `TaskHandle_t xGetNextSRTTask( void )` function, which leverages the `listGET_OWNER_OF_NEXT_ENTRY` macro to instantly access the task handle. Furthermore, the intrinsic properties of the FreeRTOS list mechanism ensure the exclusivity of a task’s state: an SRT task cannot concurrently exist in multiple state lists. If an SRT task is preempted by an HRT task boundary, the system invokes either `void vTaskTimelineSuspend( TaskHandle_t xTask )` or its interrupt-safe equivalent, safely halting the execution context without corrupting the queue integrity.

A fundamental characteristic of this implementation is that the SRT queue does not behave as a circular buffer. The execution progress is continually monitored by comparing the `ulCompletedThisFrame` counter against the `ulTotalTasks` variable within the `xSrtState` structure. Once all queued SRT tasks have completed their execution, the queue is considered depleted, and the scheduler will simply yield to the idle task during any remaining slack time until the major frame concludes.

To prepare for the subsequent deterministic cycle, the `void vResetSRTQueue( void )` function is invoked precisely at the major frame boundary. Rather than destroying and rebuilding the list—which would incur unacceptable computational overhead—this routine efficiently rewinds the internal traversal pointer, `pxIndex`, back to the anchor element, `xListEnd`. This $\mathcal{O}(1)$ operation restores the queue to its pristine initial state, ready for the next cyclic repetition.

1.8. Workload Simulation and Execution Dynamics

In order to validate the architecture, the payload function executed by both HRT and SRT tasks has been standardized. It consists of a compute-intensive algorithm based on multiply-accumulate operations, a calculation pattern extremely common in embedded systems, useful for simulating a deterministic workload. This function accepts

a single parameter, denoted as the *scaling factor*, which is directly proportional to the quantity of clock cycles required to complete its execution.

The tasks do not invoke the workload directly but interface with it through the function `void vGenericTaskWrapper( void *pvParameters )`. This wrapper handles the extraction of the scaling factor and passes it to the actual computational function, `void vSimulateEmbeddedWorkload( uint32_t ulWorkloadScalingFactor )`. The introduction of this layer of indirection decouples the computational engine from the signatures required by the kernel and the user configuration, increasing the maintainability of the codebase and simplifying the potential future injection of customized business logic. Although the adoption of a single scalar parameter and the absence of *Inter-Process Communication* (IPC) mechanisms constitute evident architectural limitations, these constraints were accepted during the design phase. The primary objective does not reside in the complexity of the payload, but rather in the robust validation of the scheduling dynamics, for which a purely computational simulation is deemed academically adequate.

Upon completion of the execution of `vSimulateEmbeddedWorkload`, the task invokes the function `void vTimelineMarkHRTComplete( void )` or `void vTimelineMarkSRTComplete( void )`, depending on its assigned nature. These routines update the global state variables pertaining to the recently concluded task, force the suspension of the task itself, and immediately trigger a *context switch* by invoking the macro `taskYIELD()`. This semi-cooperative architecture is crucial for overall performance: if an HRT task finishes its computation ahead of its deadline, the explicit yielding of control allows a prospective SRT task to take over instantaneously, without having to passively await the subsequent hardware interrupt of the system timer.

If there is available room for SRT execution, the identification of the subsequent task is delegated to the function `void vHandleSRTExecution( void )`. The latter inspects the static queue and retrieves the ready task at the head of the list, strictly implementing a FIFO policy.

1.9. Context Restoration and Cyclic Reset

At the completion of every full cycle of the major frame, the entire temporal architecture needs to be cyclically rewound. This process is governed by the function `void vResetTask( void )`, which acts as an interface towards a structural modification specifically introduced within the core files of the kernel: `void vTaskTimelineReset( TaskHandle_t xTaskToReset, TaskFunction_t pxTaskCode, void *pvParameters,`

`configSTACK_DEPTH_TYPE uxStackDepth )`.

This function exposes and manipulates the internal data structures of the operating system to perform a surgical low-level restoration. Operationally, the reset consists of repositioning the *Stack Pointer* (SP) to the base of the allocated memory area and overwriting the *Program Counter* (PC) with the original entry address of the task. This approach completely clears the execution context, preparing the task for a clean startup in the subsequent cycle. The parameter `uxStackDepth` ensures that the procedure rigorously respects the memory limits previously defined by the user configuration.

The adoption of a direct manipulation logic for the registers and the TCB, replacing the classic kernel primitives (`vTaskDelete` followed by `vTaskCreate`), represents a highly considered design choice for the optimization of CPU overhead. The standard destruction and creation functions impose heavy computational cycles and, by requiring invocations to the dynamic memory allocator, introduce non-deterministic execution times, resulting in a clear violation of real-time paradigms. Moreover, repeated cycles of allocation and deallocation inexorably lead to heap fragmentation. The `vTaskTimelineReset` function, operating in constant time on memory areas already statically allocated, eradicates these critical issues at their root, confirming and reinforcing the static-only approach of the entire project.

2. Memory Management and Static Allocation

The memory architecture of the real-time scheduler represents a critical component of its deterministic behavior. As introduced in Section 1, this section formalizes the mechanisms implemented to entirely circumvent dynamic allocation during both the initialization and execution phases.

2.1. Allocation Strategies: Dynamic vs. Static

To achieve a fully deterministic approach, our scheduler adopts strict static memory management. This approach is enabled by modifying the configuration field `configSUPPORT_STATIC_ALLOCATION` from 0 to 1 within the `FreeRTOSConfig.h` header file, and analogously changing the `configSUPPORT_DYNAMIC_ALLOCATION` field from 1 to 0. Through this configuration, the FreeRTOS *kernel* becomes aware that it cannot rely on dynamic memory, such as the heap, during runtime. Consequently, FreeRTOS removes the `heap_4.c` memory management source file from the build process, making it impossible for our system to execute dynamic allocation.

Therefore, if a user attempts to invoke the dynamic task creation function, whose full declaration is `BaseType_t xTaskCreate( TaskFunction_t pxTaskCode, const char * const pcName, const configSTACK_DEPTH_TYPE usStackDepth, void * const pvParameters, UBaseType_t uxPriority, TaskHandle_t * const pxCreatedTask )`, the system will inevitably crash or fail to compile, as `xTaskCreate` is no longer available⁴. The dynamic approach would introduce uncertainty regarding the atomicity of our scheduler, because this type of allocation is not necessarily executed in $\mathcal{O}(1)$ time complexity, but is instead subject to potential delays caused by system load and memory fragmentation.

In our specific implementation, this choice would not have been strictly critical, since task creation is performed before the scheduler is started, and the cyclic reset is not achieved by destroying and recreating all tasks. Nevertheless, the static approach was chosen to ensure strict compliance with real-time principles. Consequently, specific implementation strategies were adopted to pass all necessary information to the kernel.

2.2. Task-Specific Memory Architecture

To guarantee the stability and determinism of the static approach, three fields were added to the data structure characterizing the tasks (see Section 1.3). The first field is the stack depth variable, declared as `configSTACK_DEPTH_TYPE xStackDepth`. This variable defines the stack space available to the task for allocating local variables, calling internal functions, and saving the hardware context switch state when the task is preempted.

The second necessary component is the stack buffer pointer, declared as `StackType_t * pxStackBuffer`. This is a pointer to a statically allocated memory block, representing the exact physical memory that the task will have at its disposal for its stack. By predefining this array at compile time and passing its reference within the configuration structure, it is guaranteed that the memory is contiguous and permanently reserved. This strategy entirely prevents runtime allocation and the associated risk of memory exhaustion during execution.

Finally, the third element is the task buffer pointer, declared as `StaticTask_t * pxTaskBuffer`. This structure serves as the underlying Task Control Block. The kernel utilizes the TCB to maintain the architectural state of the task, which includes the current stack pointer, the execution state, and the pointers to internal

list nodes. Providing this buffer statically ensures that the kernel inherently possesses the necessary memory to reliably track the lifecycle of the task without dynamically requesting new resources.

3. Environment Setup and Deterministic Emulation

The physical organization of the codebase has been structured to maintain a rigorous separation of concerns between the core logic, the architectural modifications, and the analytical tools. The software architecture is reflected in a modular repository structure, designed to decouple the hardware abstraction layer, the core scheduler extensions, and the automated validation suite.

3.1. Software Architecture and Repository Layout

The following directory tree represents the actual implementation state of the project [3]:

Listing 2: Project’s directory tree.

```

-- Demo/
  -- Headers/
  -- Output/
  -- suite/
  -- Tools/
  -- FreeRTOSConfig.h
  -- integrated_test.py
  -- main.c
  -- tasks_generated.c
  -- timeline.c
  -- timeline_internal.c
  -- trace.c
  -- trace_dumper.c
  -- Makefile
-- FreeRTOS/
-- logs/
-- test_reports/
-- Dockerfile
-- README.md

```

The `Demo/` directory encapsulates the entirety of the custom logic. Key modules include `timeline_internal.c`, which manages the high-resolution dispatching of HRT tasks, and `trace_dumper.c`, a specialized task that offloads telemetry data without interfering with high-priority timing constraints.

3.2. Cross-Platform Portability and Reproducibility

The project can be built and executed both natively on Linux and in any other host environment. To guarantee full reproducibility across different machines and mitigate dependency conflicts, Docker

⁴This design choice strongly aligns with the MISRA-C:2012 Directive 4.12, which strictly prohibits the use of dynamic memory allocation in safety-critical firmware to prevent heap exhaustion and memory leaks.

was adopted as the primary build and execution environment.

The `Dockerfile` defines a consistent environment containing the `arm-none-eabi` cross-compiler and the QEMU emulator configured for the Cortex-M3 architecture. The `integrated_test.py` script orchestrates this environment by launching ephemeral containers that build the firmware, execute the scheduled workload, and systematically extract the trace data. This robust architecture ensures that the observed jitter (≤ 1 tick) and scheduler overhead ($\leq 10\%$) remain mathematically consistent regardless of the developer’s native operating system.

3.3. Virtual Clock Configuration and Determinism

To guarantee execution reproducibility across different host machines, the build system configures the QEMU emulator utilizing the `-icount` parameter. Without this specific configuration, the FreeRTOS kernel would execute instructions as rapidly as the host CPU allows. Consequently, the real-time duration of a major frame would fluctuate unpredictably based on the host’s instantaneous computational load.

The `-icount` flag, coupled with the `sleep=on` sub-parameter, strictly links the virtual time to the number of executed instructions, preventing the virtual clock from advancing faster than the real-time clock and thereby ensuring highly deterministic analytical results. Specifically, the emulator is invoked with the `-icount shift=6,align=off,sleep=on` directive. In QEMU, the `shift` parameter specifies that the virtual CPU advances the virtual time by 2^{shift} nanoseconds per executed instruction. A shift value of 6 yields 64 nanoseconds per instruction, which effectively simulates a virtual clock frequency of approximately 15.625 MHz⁵.

This constitutes a deliberate underestimation of the physical target board, which operates at 25 MHz as defined by the `configCPU_CLOCK_HZ` macro. While a shift value of 5 (yielding 32 nanoseconds per instruction, or 31.25 MHz) would theoretically be closer to the actual hardware frequency, it would inherently overestimate the target’s capabilities. Selecting 15.625 MHz enforces a strictly conservative approach: if the timeline scheduler meets all strict temporal constraints within this degraded virtual environment, its robust compliance on the actual 25 MHz silicon is rigorously guaranteed.

Finally, it is imperative to acknowledge that QEMU operates as an *Instruction Set Simulator* (ISS) rather than a cycle-accurate emulator. The virtualization layer cannot differentiate between

single-cycle atomic operations and complex multi-cycle instructions, nor does it model hardware-level latencies such as memory wait states or pipeline stalls. As a result, the jitter metrics extracted in this environment measure *algorithmic jitter*—temporal variations induced purely by differing software execution paths—rather than physical hardware jitter. Nevertheless, successfully demonstrating that this algorithmic variance remains tightly bounded (e.g., ≤ 1 tick) rigorously validates the $O(1)$ deterministic logic of the proposed scheduling architecture.

4. Kernel Integration and Scheduling Hooks

The realization of a time-triggered scheduling engine on top of FreeRTOS requires deliberate and carefully scoped modifications to the kernel internals. Rather than replacing the kernel wholesale, the adopted strategy consists of extending it at precisely defined intervention points. This preserves portability and licensing compliance whilst successfully injecting the deterministic behavior demanded by an HRT execution model. The following subsections detail each architectural layer of this integration, from the low-level task-state manipulation primitives exposed by `tasks.c`, through the tick-driven dispatch engine, to the deadline-enforcement and context-switching policy.

4.1. Execution State Management and Transition

To ensure that tasks strictly adhere to their assigned temporal boundaries, the system introduces direct suspension and resumption mechanisms. The function `void vTaskTimelineSuspend( TaskHandle_t xTask )` is employed to remove a task from the standard execution lists, thereby mandating the cessation of execution outside its designated minor frame or following the explicit completion of its workload. Should this operation need to be executed within an ISR, the system relies on the variant `void vTaskTimelineSuspendFromISR( TaskHandle_t xTaskToSuspend )`. This version is primarily utilized to handle the forced termination of HRT tasks that exceed their deadline, applying the immediate termination policy, or for preemption management.

Symmetrically, the activation of a task based on static scheduling is delegated to the function `void vTaskTimelineResume( TaskHandle_t xTask )`, which forces the task into an executable state, thereby circumventing priority evaluation. The corresponding interrupt-safe function, `void vTaskTimelineResumeFromISR( TaskHandle_t`

⁵The calculation $2^6 = 64$ nanoseconds per instruction translates to an effective virtual clock rate of $1/(64 \times 10^{-9}) \approx 15.625$ MHz.

`xTaskToResume`), is invoked exactly when the scheduled start time boundary of an HRT task is reached. To verify a task’s status programmatically, the functions `BaseType_t xTaskIsSuspended( TaskHandle_t xTaskToCheck )` and its counterpart `BaseType_t xTaskIsSuspendedFromISR( TaskHandle_t xTaskToCheck )` are provided, returning a boolean value indicative of whether the specified task is currently suspended.

4.2. Execution Flow Interception and Context Switching

Deterministic control over time progression and task selection is exerted via two fundamental hooks integrated directly into the core of the kernel. The function `BaseType_t xTimelineTickHook( void )` is invoked within the system tick increment routine. This component continuously evaluates the current global tick against the predefined frame architecture. Its role is twofold: on one hand, it identifies deadline misses for HRT tasks, and on the other, it determines when a forced context switch is mandatory to preserve the strict repeatability of the temporal sequence. If the current execution requires an interruption to maintain temporal consistency, `xTimelineTickHook` returns a value that compels the kernel to execute a context switch.

Subsequently, during the task selection phase, the function `TaskHandle_t xTimelineScheduleHook( void )` intervenes, being deeply embedded within the context switch routine. This module subverts the standard priority-based selection mechanism, rigidly applying the principles of the TT paradigm. It ensures that HRT tasks are executed non-preemptively exactly within their allocated time slots. During periods when no HRT task is scheduled, `xTimelineScheduleHook` manages pending SRT tasks following a FIFO policy. If a task managed by the timeline scheduler is ready for execution, the function returns its handle; otherwise, it returns a null pointer, signaling the system to yield control to the native system idle task or to revert to the standard scheduler.

4.3. Deterministic Dispatching and Amortized Access

The core of the timeline-driven scheduling logic is centralized within the two primary kernel hooks discussed previously. These functions are fundamentally responsible for enforcing temporal constraints and selecting the subsequent task to execute, entirely circumventing the priority mechanisms of the operating system.

The `xTimelineTickHook` function operates through a multi-phase evaluation pipeline. The

first phase focuses on deadline enforcement: it performs a strict $\mathcal{O}(1)$ check to determine whether the currently active HRT task has reached its allocated end time. If this temporal boundary is violated, the task is immediately suspended to prevent cascading failures across the schedule.

Subsequently, the hook executes the HRT activation evaluation to select the next task slated for execution. A critical design requirement was to efficiently retrieve the next HRT task from the statically allocated and chronologically sorted array of `TimelineTaskConfig_t` structures without incurring the computational overhead of a linear $\mathcal{O}(N)$ scan at every tick. During the architectural design phase, the implementation of alternative data structures, such as lookup tables or hash maps, was carefully considered to mathematically guarantee a strict $\mathcal{O}(1)$ access time. However, such solutions would have introduced undesired dynamic memory allocations and significantly increased the structural complexity of the kernel.

Consequently, an optimized approach was adopted leveraging an internal state variable, `ulHrtTaskIndex`. Because the task configuration array is securely sorted during the initialization phase, the scheduler utilizes this index to memorize the position of the next potential HRT activation. The evaluation loop strictly commences from `ulHrtTaskIndex`; once a task’s start time coincides with the current system tick, the task is authorized to execute, and the index is updated to the newly discovered position. This strategic design choice yields a practically amortized $\mathcal{O}(1)$ access time, maximizing kernel performance while maintaining implementation simplicity and preserving a zero-allocation footprint.

Following the HRT evaluation, if no HRT task is active or if the scheduled task completes prematurely, `xTimelineTickHook` evaluates the SRT queue to reclaim available idle time. Finally, the function checks for major frame boundaries to trigger cyclic system resets, returning a boolean flag to indicate whether a forced context switch is required.

To conclude the scheduling cycle, the `xTimelineScheduleHook` function acts as the authoritative task resolver during a context switch. It operates by evaluating the hierarchical state of the scheduler. It primarily verifies the HRT execution state; if an HRT task is active and not suspended, its handle is returned immediately, granting it absolute dominion over the processor. In the absence of HRT authority, the hook evaluates the SRT state, returning the preempted or next-in-line SRT task if one is available. If no timeline-managed tasks are eligible for execution, the function securely returns a null pointer.

5. Telemetry System and Asynchronous Tracing

The tracing module constitutes a fundamental component of the system, acting as the primary telemetry conduit responsible for recording and outputting scheduling information pertaining to the executed tasks. This section outlines the asynchronous mechanisms employed to extract granular performance data without violating real-time execution constraints.

5.1. Asynchronous Event Logging and Buffer Management

This mechanism is intrinsically event-based, designed to save the state of the system directly into a *Random-Access Memory* (RAM) structure, specifically utilizing a circular buffer. The primary rationale behind this architectural choice is to strictly avoid the execution of blocking operations, such as UART transmissions, within critical sections of the kernel or inside an ISR. By decoupling the high-frequency event logging from the slow input/output operations, the module ensures that the printing of information occurs in a strictly asynchronous manner. This non-blocking approach represents a core strength of the architecture, as it preserves the real-time determinism required by the system, ensuring that temporal constraints are never violated by telemetry overhead.

The recording of individual events is performed through the function `void vTraceLog( uint32_t ulMajorFrameTimestamp, uint8_t ucEventID, uint32_t ulData )`. Whenever a relevant scheduling event occurs, such as a task start or a deadline miss, `vTraceLog` inserts a lightweight structured record directly into the RAM buffer.

Subsequently, a dedicated consumer task, defined by the `void vTraceDumperTask( void *pvParameters )` signature, retrieves these records asynchronously. The `vTraceDumperTask` extracts the raw binary data and formats it into human-readable strings before transmitting them over the UART interface. This multi-stage pipeline ensures that the computationally heavy string formatting and serial transmission delays do not impact the active task execution or the ISR timing profiles.

5.2. Offline Analysis and Telemetry Integration

The asynchronous telemetry data printed by the tracing module enables the user to perform continuous system monitoring during runtime. Furthermore, the generated log files serve as the robust foundation for an exhaustive offline evaluation, the details of which are expounded in Section 6. A dedicated Python script, named `integrated_test.py`,

is designed to automatically parse the information printed in the serial log. This script executes a comprehensive suite of analysis operations, providing a detailed assessment of the executed scheduling configuration, calculating precise metrics, and formally verifying the overall stability and correctness of the timeline execution.

6. Validation Framework and Automated Testing

To rigorously validate the developed scheduler against the strict non-functional constraints, a comprehensive testing framework was established. This section delineates the automated validation ecosystem designed to empirically verify the correctness, performance, and boundary limits of the scheduling architecture.

6.1. Pipeline Orchestration and Test Execution

The project incorporates an automated testing framework implemented in Python, physically centered around the script `integrated_test.py`. This component orchestrates the entire validation pipeline of the timeline scheduler implementation, systematically covering build execution, workload deployment, log acquisition, performance evaluation, and regression checking.

The framework is structured around a strict sequential workflow: the system is first compiled and linked, either natively on Linux or inside a Docker container; the resulting firmware is then deployed on the QEMU emulator and executed; the UART output is captured as a raw log; scheduling events are extracted and parsed; performance metrics are computed from the parsed data; pass/fail regression checks are evaluated against expected properties; and, optionally, graphical visualizations of the scheduling behavior are produced. The infrastructure is thoughtfully organized into multiple Python modules to enforce separation of concerns and maximize code maintainability.

The script `integrated_test.py` constitutes the single entry point of the automated testing pipeline. It is inherently responsible for parsing command-line arguments, selecting the active test configuration, invoking the build system through `make`, launching QEMU, and safely handling execution lifecycle events, including graceful degradation upon user interruption with the preservation of partial logs.

Upon successful execution, the orchestrator triggers log parsing and analysis, subsequently producing structured test reports. Multiple configurations are supported natively, enabling automated regression testing across distinct scheduling work-

loads; each configuration is described by a JSON file specifying the workload parameters to be statically injected at compile time.

6.2. Log Parsing and Frame Boundary Detection

The file `log_parser.py` is strictly responsible for extracting structured information and statistical data from the raw UART log produced by the QEMU emulator.

Prior to parsing, an internal routine removes any incomplete major frame appended to the log as a consequence of an interrupted execution, thereby preventing corrupted or truncated scheduling data from propagating into the statistical analysis. Major frame boundaries are subsequently identified by two complementary mechanisms: the explicit detection of the log marker, and the algorithmic detection of tick wrap-around events, where the major frame tick counter resets to zero. The maximum of the two counts is retained to mathematically guarantee robust frame detection even in the presence of missing log lines.

The parser identifies and tallies all relevant scheduling events emitted by the trace module. Specifically, it tracks task starts, completions, preemptions, forced terminations, and deadline misses. Regular expressions are employed to selectively extract, for each major frame, the idle tick statistics, the kernel overhead expressed in both cycles and percentage, and the maximum and minimum release delay values recorded for each individual HRT activation.

6.3. Metrics Extraction and Jitter Analysis

The file `models.py` defines a specific data structure which aggregates all statistics produced during parsing. The structure consolidates event counters, idle tick samples, kernel overhead samples, and per-task delay statistics, ensuring that the parsing logic and the statistical representation remain fully decoupled. This rigid separation allows the metrics model to be consumed independently by the regression checking engine, the report generator, and the visualization module.

The analytical suite utilizes the parsed data model to derive high-level indicators of system stability and real-time compliance. The framework computes, for each test execution, the total number of HRT activations, HRT completions, and deadline misses, alongside SRT activations, completions, and preemption events. Major frame completions are counted and verified separately. Idle time and kernel overhead are both characterized statistically through their mean and standard deviation calculated across all observed major frames.

For each HRT task, release jitter is formally defined as the absolute difference between the maximum and minimum measured release delays observed across multiple execution cycles:

$$\text{Jitter} = \text{delay}_{\max} - \text{delay}_{\min}$$

The framework computes the average and maximum jitter per task, as well as the global maximum jitter across all HRT activations in the session. These specific quantities provide a quantitative assessment of temporal predictability and allow definitive verification of the requirement that jitter does not exceed one system tick under any workload condition.

6.4. Visualization Generation and Regression Checking

The file `visualization.py` is dedicated to the generation of visual artifacts to aid in the manual inspection of the scheduling profiles.

A graphical timeline plot is generated to visualize task execution over the duration of a major frame. HRT tasks are rendered in varying shades of orange, while SRT tasks are assigned distinct shades of blue; within each category, specific tasks receive progressively differentiated hues to preserve readability in configurations with a high task count. Idle intervals are reconstructed mathematically by identifying periods during which no task is reported as active in the parsed event sequence.

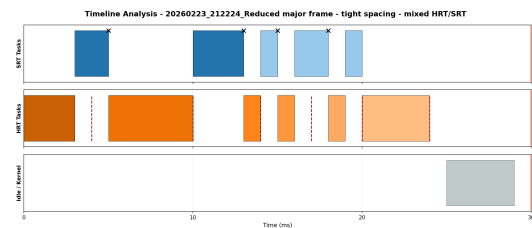


Figure 1: Timeline diagram of a representative major frame execution.

A bar chart summarizes the total execution time attributed to each task across the observed frames, offering immediate visual insight into workload balance and scheduling fairness. Kernel overhead is explicitly represented as a distinct bar to allow its contribution to be directly compared against the 10% computational budget imposed by the project requirements.

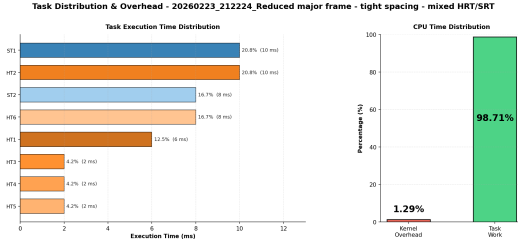


Figure 2: Task execution distribution and kernel overhead summary.

Automated regression checks evaluate the correctness and stability of the scheduler against a set of expected properties encoded within each JSON configuration file. Verifiable properties include the strict absence of deadline misses, the expected number of HRT activations per major frame, and the absolute stability of kernel overhead across sequential frames⁶. Each check produces a labelled pass or fail verdict accompanied by a human-readable diagnostic detail, enabling traceable and scientifically reproducible validation across the entire test suite.

7. Concluding Remarks and Final Assessment

The automated test infrastructure constitutes the primary empirical instrument through which the correctness and real-time compliance of the scheduler have been formally established. The regression suite confirmed the absolute temporal predictability of the scheduling engine. Rather than evaluating successful task completion—which is intrinsically dependent on user-defined workloads and configurations—the validation rigorously demonstrated activation determinism. The empirical data proved that HRT tasks are consistently released at their exact scheduled instants across every cyclic repetition, exhibiting the highly predictable behavior characteristic of a true real-time system. This strict determinism is quantitatively supported by the jitter analysis, which demonstrated that the release variance remains bounded within one system tick for all tested configurations, satisfying the primary non-functional requirement. Similarly, kernel overhead measurements, computed by accumulating cycles consumed within both the `xTimelineTickHook` and the context switch routines, remained consistently below the 10% CPU budget across all workload profiles.

The test configurations encompassed scenarios with tight HRT slot spacing to stress boundary enforcement, SRT preemption under high utilization,

⁶The kernel overhead is computationally measured by sampling the Cortex-M3 SysTick, a 24-bit decrementing hardware counter, at the entry and exit points of the kernel evaluation hooks.

and nominal cyclic repeatability. The visualization artifacts proved particularly valuable, corroborating analytical metrics with graphical confirmation of deterministic behavior. Collectively, the results validate the architectural choices made throughout the project—in particular, the static-only memory model, the amortized $\mathcal{O}(1)$ HRT activation search, and the low-level context restoration—as sound and effective solutions to the inherent tensions between determinism, performance, and implementation complexity in a resource-constrained real-time environment.

References

- [1] FreeRTOS. *FreeRTOS — Documentation*. Documentation Overview of Real-time operating system for microcontrollers and small microprocessors. 2026. URL: <https://www.freertos.org/Documentation/00-Overview>.
- [2] Politecnico di Torino. *Project 1 — Precise Scheduler for FreeRTOS*. Course: Operating Systems for Embedded Systems (EOS). 2026. URL: <https://baltig.polito.it/teaching-material/02npsov-operating-systems-for-embedded-systems/EOS/-/blob/main/Project-Assignment-2026.md>.
- [3] Group 49. *GitLab — Repository of Group 49*. Authors: Lombardozzi, Christian; Masoumkhani, Amin; Papantuono, Federico; Pedron, Marco; Stella, Paolo. 2026. URL: <https://baltig.polito.it/eos25/group49>.